

CAS4160 Homework 6: Model-based RL

[Your name]
[Your student ID]

1 Introduction

이 과제의 목표는 모델 기반 강화학습(model-based reinforcement learning)에 대한 경험을 쌓는 것입니다. 일반적으로 모델 기반 강화학습은 두 가지 주요 부분으로 구성됩니다: (1) 관찰된 상태 전이(state transitions)를 모델링하기 위한 동역학 함수(dynamics function)를 학습하는 것, 그리고 (2) 학습된 모델의 예측을 이용하여 어떤 행동을 취할지 결정하는 것입니다. 모델 예측은 예측된 보상을 최대화하는 행동을 직접 찾거나, 정책(policy)을 학습하는 데 사용될 수 있습니다.

이 과제에서는 학습된 동역학 모델을 이용하여 행동을 계획(plan)하는 것과 함께 동역학 모델을 학습하는 것을 구현하게 됩니다. 특히, 명시적인 정책 없이 모델을 통한 계획만으로 행동을 생성하는 CEM을 구현하게 됩니다.

2 Code structure overview

코드 구조는 이전 과제와 유사합니다. 이번 과제에서는 주로 다음 코드들을 다루게 됩니다:

- `cas4160/scripts/run_hw6.py`: the main training loop for your implementation.
- `cas4160/agents/model_based_agent.py`: the learner you will implement.

3 Model-based reinforcement learning

모델 기반 강화학습(MBRL)에 대한 간단한 개요를 제공하며, 이번 과제에서 구현하게 될 특정 유형의 MBRL에 대해 설명합니다.

MBRL은 주로 두 부분으로 구성됩니다: (1) 동역학 모델 학습(Section 3.1)과 (2) 학습된 동역학 모델을 이용하여 보상 함수를 최대화하는 행동을 계획하고 실행하는 것(Section 3.2).

3.1 Learning dynamics model

이번 과제에서는 다음 형태의 신경망 동역학 모델 f_θ 를 학습합니다:

$$\delta_{t+1} = f_\theta(\mathbf{s}_t, \mathbf{a}_t), \quad (1)$$

이 모델은 현재 상태와 행동이 주어졌을 때 상태의 변화를 예측합니다. 따라서 예측된 δ_{t+1} 을 이용하여 다음 상태 예측을 다음과 같이 생성할 수 있습니다:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \delta_{t+1}. \quad (2)$$

왜 네트워크가 다음 상태를 직접 예측하는 대신 상태 차이를 예측하도록 하는 것이 좋은지에 대한 직관은 이 [논문](#)을 참조하세요. f_θ 는 다음과 같은 목적 함수에 대한 경사 하강법을 수행하는 표준 지도 학습 설정으로 학습됩니다:

$$L(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) \in \mathcal{D}} [\|(\mathbf{s}_{t+1} - \mathbf{s}_t) - f_\theta(\mathbf{s}_t, \mathbf{a}_t)\|_2^2]. \quad (3)$$

실제로는 신경망의 타겟 값을 정규화하는 것이 도움이 됩니다. 따라서 우리는 상태 변화의 정규화된 버전을 예측하도록 네트워크를 학습합니다:

$$L(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) \in \mathcal{D}} [\|\text{Normalize}(\mathbf{s}_{t+1} - \mathbf{s}_t) - f_\theta(\mathbf{s}_t, \mathbf{a}_t)\|_2^2]. \quad (4)$$

f_θ 는 정규화된 상태 차이를 예측하도록 학습되므로, 다음 상태는 다음과 같이 예측할 수 있습니다:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \text{Unnormalize}(f_\theta(\mathbf{s}_t, \mathbf{a}_t)). \quad (5)$$

간단하게 $F_\theta(\mathbf{s}, \mathbf{a}) = \mathbf{s} + \text{Unnormalize}(f_\theta(\mathbf{s}, \mathbf{a})) = \mathbf{s}'$ 로 정의합니다.

3.2 Action optimization

학습된 동역학 모델 f_θ 가 주어지면, 이제 알려진 보상 함수를 최대화하는 행동을 선택하고 실행하고자 합니다. 최적의 행동은 다음 최적화 문제를 풀어 찾을 수 있습니다:

$$\mathbf{a}_t = \operatorname{argmax}_{\mathbf{a}_{t:\infty}} \sum_{t'=t}^{\infty} r(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}), \text{ where } \hat{\mathbf{s}}_{t'+1} = F_\theta(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}) \text{ and } \hat{\mathbf{s}}_t = \mathbf{s}_t. \quad (6)$$

그러나 Equation (6)을 푸는 것은 두 가지 이유로 비실용적입니다: (1) 무한한 행동 시퀀스에 대해 계획하는 것은 불가능하고, (2) 학습된 동역학 모델은 완벽하지 않기 때문에 이를 이용해 개방 루프(open-loop) 방식으로 계획하면 시간이 지남에 따라 오류가 누적되어 먼 미래까지 계획하는 것이 매우 부정확해집니다. 대신, 다음과 같은 유한 시퀀스, 유한 길이의 최적화 문제를 풀 수 있습니다:

$$A^* = \operatorname{argmax}_{\{A^{(0)}, \dots, A^{(K-1)}\}} \sum_{t'=t}^{t+H-1} r(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}), \text{ where } \hat{\mathbf{s}}_{t'+1} = F_\theta(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}) \text{ and } \hat{\mathbf{s}}_t = \mathbf{s}_t, \quad (7)$$

여기서 $A^{(k)} = (\mathbf{a}_t^{(k)}, \dots, \mathbf{a}_{t+H-1}^{(k)})$ 는 길이 H 의 무작위 행동 시퀀스입니다. Equation (7)가 의미하는 것은 길이 H 의 K 개 무작위 행동 시퀀스를 고려하고, 각 행동 시퀀스를 취했을 때의 결과를 학습된 동역학 모델 f_θ 를 사용해 예측(즉, 미래 상태)한 후, 각 후보 행동 시퀀스에 대한 보상을 평가하여 가장 좋은 행동 시퀀스를 선택하는 것입니다. 이 접근 방식은 미래 H 스텝까지만 계획하므로 모델 오류의 누적을 방지할 수 있다는 장점이 있지만, 장기 과제를 해결하기에는 충분하지 않을 수 있다는 한계도 있습니다.

이 무작위 사격(random-shooting) 최적화 접근법보다 더 나은 대안은 교차 엔트로피 방법(Cross-Entropy Method, CEM)입니다. CEM은 랜덤 슈팅과 유사하지만, 샘플링되는 행동 시퀀스의 분포를 반복적으로 개선합니다. 먼저 랜덤 슈팅과 마찬가지로 K 개의 행동 시퀀스 $\{A^{(0)}, \dots, A^{(K-1)}\}$ 를 무작위로 초기화합니다. 그 다음, 예측된 할인 보상의 합이 가장 높은 J 개의 시퀀스를 “엘리트(elite)” 행동 시퀀스로 선택합니다. 이후 엘리트 행동 시퀀스와 동일한 평균과 분산을 가지는 대각 가우시안 분포를 피팅하고, 이를 다음 반복에서의 행동 샘플링 분포로 사용합니다. 이 과정을 M 번 반복한 후, 최종 가우시안의 평균을 최적화된 행동 시퀀스로 취합니다.

또한, 우리의 모델이 불완전하고 계획대로 모든 것이 완벽하게 진행되지 않기 때문에, 모델 예측 제어(Model Predictive Control, MPC) 접근법을 채택합니다. 이는 매 타임스텝마다 랜덤 슈팅 또는 CEM을 수행하여 최적의 H 스텝 행동 시퀀스를 선택하지만, 해당 시퀀스에서 첫 번째 행동만 실행하고 다음 타임스텝에서 업데이트된 상태 정보를 이용해 다시 계획하는 방식입니다. 이는 근사 동역학 모델을 사용하 먼 미래까지 계획할 때 발생하는 복잡 오류의 영향을 줄여줍니다.

3.3 On-policy data collection

MBRL은 이론적으로는 오프-폴리시(off-policy)입니다(즉, 어떤 데이터로부터도 학습할 수 있음). 그러나 실제로는 온-폴리시(on-policy) 데이터가 없으면 성능이 크게 저하됩니다. 즉, 모델이 무작위로 수집된 데이터만으로 학습된다면, 대부분의 경우 우리가 실제로 관심을 가지는 상태 공간의 부분들을 충분히 설명하지 못합니다. 따라서 반복적인 알고리즘에서 온-폴리시 데이터 수집을 활용하여 전체 과제 성능을 향상시킬 수 있습니다.

이는 다음과 같이 요약됩니다:

3.4 Ensembles

예측 성능을 개선하는 간단하고 효과적인 방법은 모델의 앙상블(ensemble)을 사용하는 것입니다. 아이디어는 간단합니다: 하나의 네트워크 f_θ 만 학습하는 대신, 독립적으로 초기화된 N 개의 네트워크 $\{f_\theta^i\}_{i=1}^N$ 를 학습합니다.

Algorithm 1 온-폴리시 데이터 수집을 이용한 모델 기반 RL

Input: 기본 정책 $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (예: 무작위 정책)을 실행하여 $D = \{\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}\}$ 수집

```
1: while 수렴하지 않을 때까지 do
2:   // 모델 롤아웃을 사용하여 데이터셋 확장
3:    $D$ 를 사용하여  $f_\theta$  학습 (Equation (4))
4:    $\mathbf{s}_t \leftarrow$  현재 상태
5:   for  $M$  번 do
6:     for  $t = 0, \dots, T$  do
7:        $A^* = \pi_{\text{MPC}}(\mathbf{a}_t | \mathbf{s}_t)$ , 여기서  $\pi_{\text{MPC}}$ 는 랜덤 슈팅 또는 CEM으로부터 얻음
8:        $\mathbf{a}_t \leftarrow A^*$ 의 첫 번째 행동
9:       환경에서  $\mathbf{a}_t$ 를 실행하여  $\mathbf{s}_{t+1}$  획득
10:       $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$ 를  $D$ 에 추가
```

테스트 시에는 각 후보 행동 시퀀스에 대해 N 개의 독립적인 롤아웃을 생성하고, 이 롤아웃들의 보상을 평균하여 가장 좋은 행동 시퀀스를 선택합니다.

4 Learning dynamics model

이 섹션에서는 무작위 행동을 실행하여 수집된 데이터셋을 사용하여 모델을 학습합니다. 구체적으로, Section 3.1에서 언급한 대로 정규화를 적용한 동역학 모델의 앙상블을 학습합니다. 모델 학습의 일반적인 관행으로는 환경의 보상을 함께 학습하는 것도 있지만, 이번 과제에서는 단순화를 위해 동역학만 학습하고 실제(true) 보상 함수를 사용합니다.

4.1 Implementation

모델 학습 코드를 `run_hw6.py`와 `model_based_agent.py`에서 구현해 봅시다:

- `cas4160/scripts/run_hw6.py`: 모든 TODO를 구현하세요.
- `cas4160/agents/model_based_agent.py`: `get_loss()`를 구현하세요.

4.2 Experiments

- Halfcheetah 환경에서 `experiments/mpc/halfcheetah_0_iter.yaml` 설정으로 모델을 학습하세요.

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_0_iter.yaml
```

- 이 설정은 동역학 모델의 앙상블만 학습합니다. 따라서 정책 평가는 수행되지 않습니다. 이 실험에서는 iteration 500까지 모델 손실이 0.2 이하로 내려가야 합니다.
- 모델 손실을 플롯하세요 (로그 디렉토리 내부의 `itr_0.loss_curve.png`를 사용하세요. `tensorboard`가 아닙니다.).

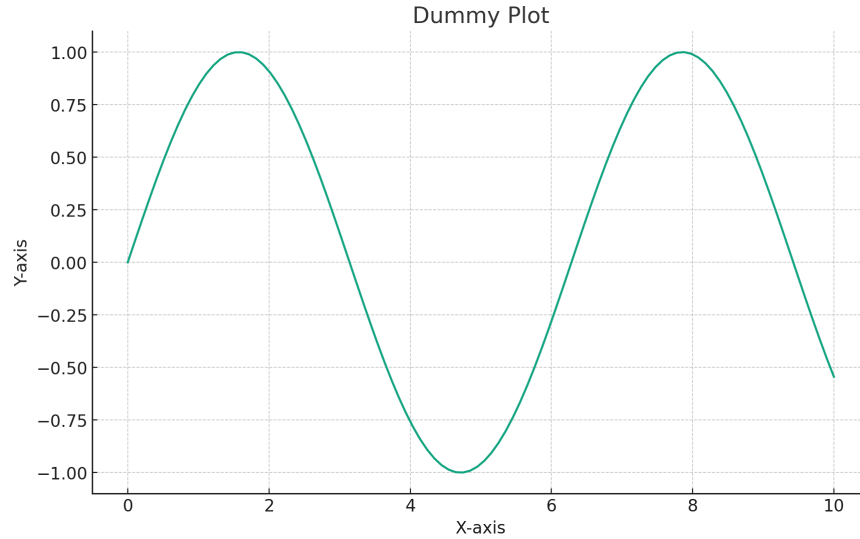


Figure 1: 이 그림을 여러분의 플롯으로 교체하고, 캡션을 추가하세요.

5 Random shooting and CEM

5.1 Implementation

이 섹션에서는 MPC를 random shooting과 CEM 두 가지 방법으로 구현합니다.

- `cs285/agents/model_based_agent.py`: 모든 TODO를 구현하세요.

5.2 Experiments

- random shooting으로 수집된 온-폴리시 데이터를 사용하여 Halfcheetah 환경에서 `halfcheetah_multi_iter.yaml` 설정으로 동역학 모델을 학습하세요. 보상이 **300 이상**이어야 합니다. (사실 모든 구현 세부사항이 정확하다면 보상은 800 이상이 됩니다. 그러나 이러한 세부사항이 누락되어도 점수는 감점하지 않습니다.)

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_multi_iter.yaml
```

- CEM으로 수집된 온-폴리시 데이터를 사용하여 Halfcheetah 환경에서 `halfcheetah_cem.yaml` 설정으로 동역학 모델을 학습하세요. 보상이 **800 이상**이어야 합니다. (random shooting과 마찬가지로, 모든 구현 세부사항이 정확하다면 보상은 1100 이상이 됩니다. 그러나 이러한 세부사항이 누락되어도 점수는 감점하지 않습니다.)

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem.yaml
```

- CEM 실험은 완료까지 약 2시간이 소요됩니다. RTX 3090을 사용 중이라면 GPU 활용도를 극대화하기 위해 다음 문제의 스크립트를 동시에 실행하는 것을 강력히 권장합니다. 그렇지 않으면 모든 실험을 완료하는 데 6시간이 걸릴 수 있습니다.
- random shooting과 CEM에 대한 결과(평균 return의 평균(mean)과 표준편차(std.))를 보고하세요. 결과를 비교하고 설명하세요.

답변:

Table 1: random shooting과 CEM의 성능. 데이터를 채우고 제목을 교체하세요.

	Mean	Standard Deviation
Random shooting	-	-
CEM	-	-

설명을 추가하세요.

- Halfcheetah 환경에서 다른 하이퍼파라미터를 적용한 CEM 에이전트를 학습하세요. 구체적으로 $H = 1$, $K = 50$ 으로 아래 명령어를 실행하여 코드를 실행하세요 (원래 값은 $H = 15$, $K = 1000$ 입니다).

```
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem_H-1.yaml
python cas4160/scripts/run_hw6.py -cfg experiments/mpc/halfcheetah_cem_K-50.yaml
```

- 3가지 설정(원래 설정, $H = 1$, $K = 50$)에 대한 CEM 결과(평균 return의 평균과 표준편차)를 보고하세요. 결과를 비교하고 설명하세요.

답변:

Table 2: 3가지 다른 하이퍼파라미터를 적용한 CEM의 성능. 데이터를 채우고 제목을 교체하세요.

	Mean	Standard Deviation
CEM	-	-
CEM ($H = 1$)	-	-
CEM ($K = 50$)	-	-

설명을 추가하세요.

6 Submission

Please submit the code and the “report” in a single zip file, hw6_[YourStudentID].zip. The zip file should be smaller than 50MB. The structure of the submission file should be:

```
hw6_[YourStudentID].zip
├── hw6_[YourStudentID].pdf
├── cas4160
│   ├── agents
│   │   └── model_based_agent.py
│   └── ...
└── ...
```